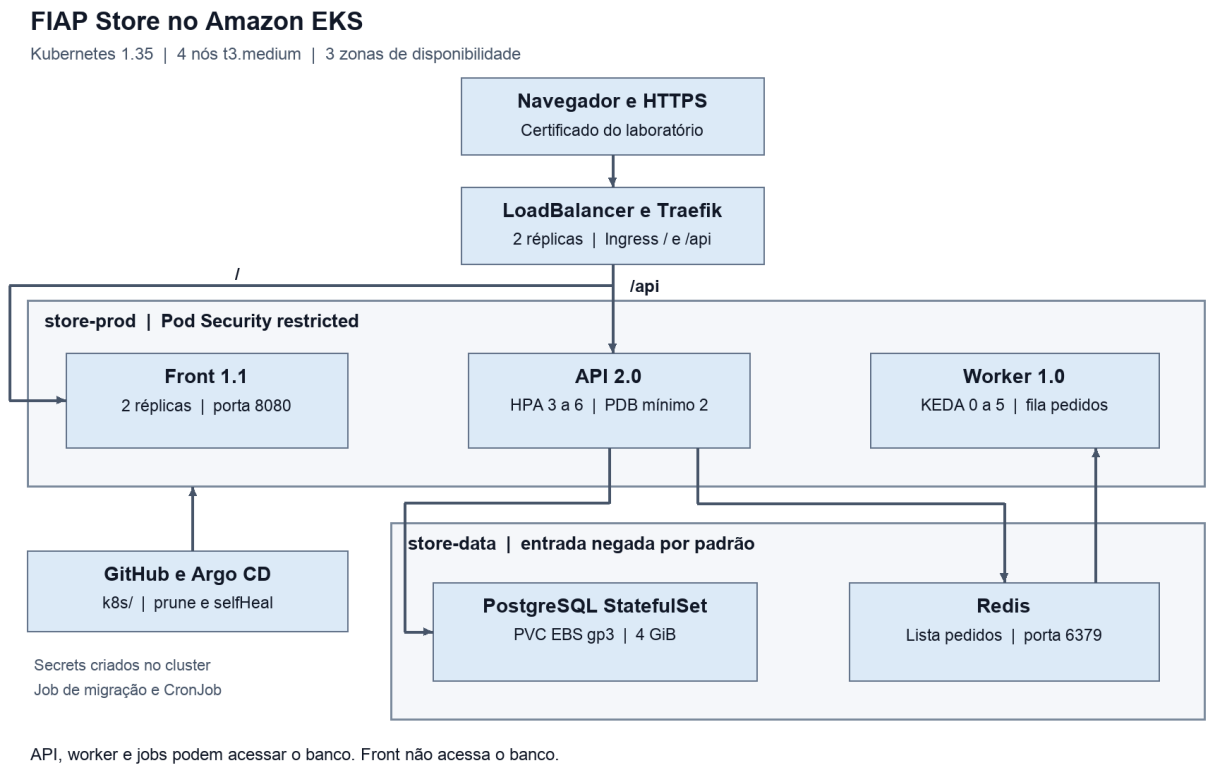
FIAP Store no Amazon EKS

Relatório da arquitetura e das evidências

Ronaldo Madukaife John Attamah | RM 564630 | 6 de novembro de 2026

A FIAP Store foi implantada no AWS Academy com entrega por GitOps. Nesta sessão, os dez desafios foram ensaiados com sucesso e a avaliação automática confirmou 80 de 80 pontos. As capturas mostram recursos reais do ambiente; os logs completos estão no repositório.



API, worker e jobs podem acessar o banco. Front não acessa o banco.

Componente	Configuração validada
EKS	Kubernetes 1.35; quatro nós t3.medium em três zonas
Aplicação	API 2.0 com três réplicas; front 1.1 com duas; worker em zero sem fila
Dados	PostgreSQL StatefulSet com PVC EBS gp3 de 4 GiB; Redis interno
Entrega	GitHub MaduKa01/fiap-store; Argo CD com prune e selfHeal

O banco tem uma instância e volume em uma zona. Sua recriação preserva dados, mas pode interromper a API temporariamente; não há failover automático do banco entre zonas.

Infraestrutura e capacidade

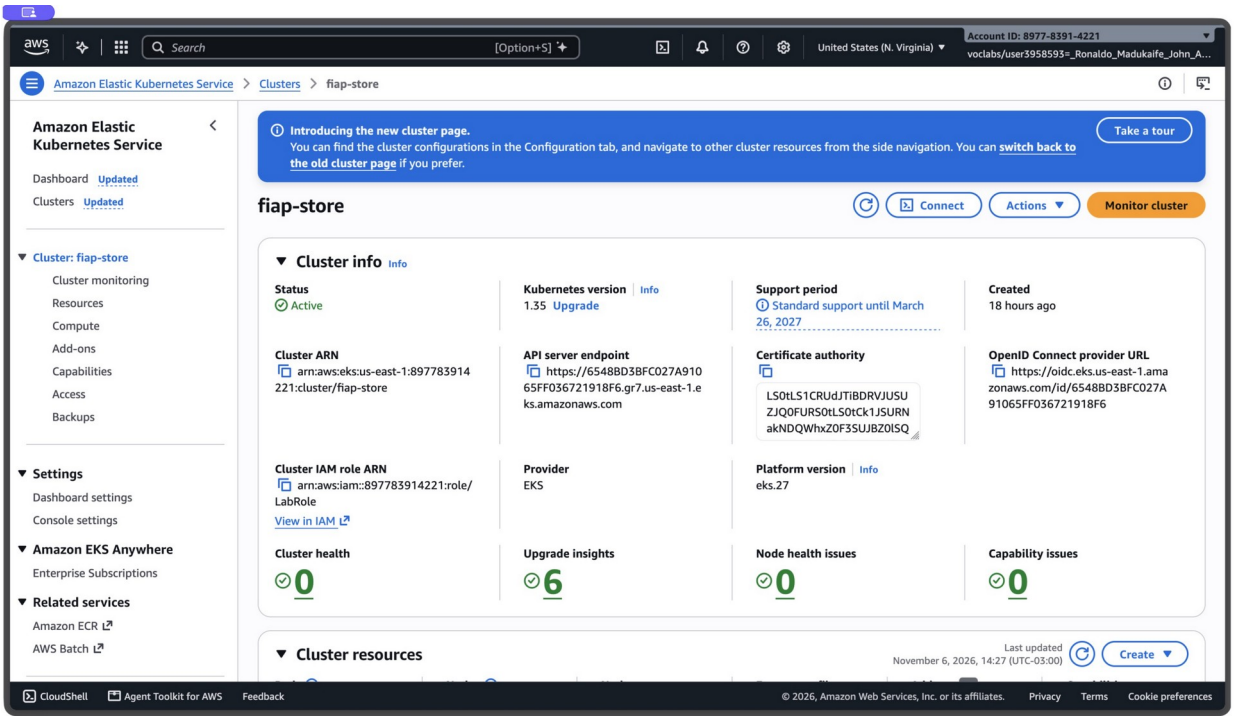


Figura 1 Cluster fiap-store ativo no console AWS, em us-east-1, com Kubernetes 1.35.

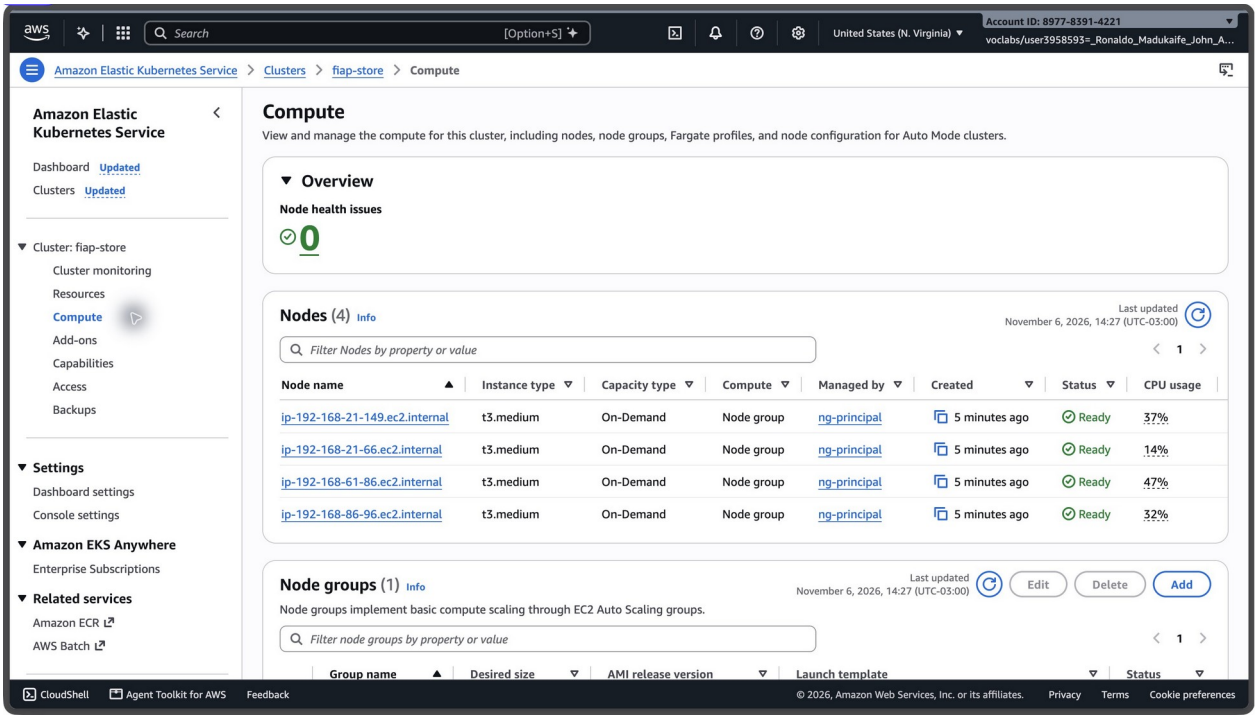


Figura 2 Quatro nós t3.medium Ready. A distribuição em us-east-1a, us-east-1b e us-east-1c também foi conferida com kubectl.

Aplicação e armazenamento

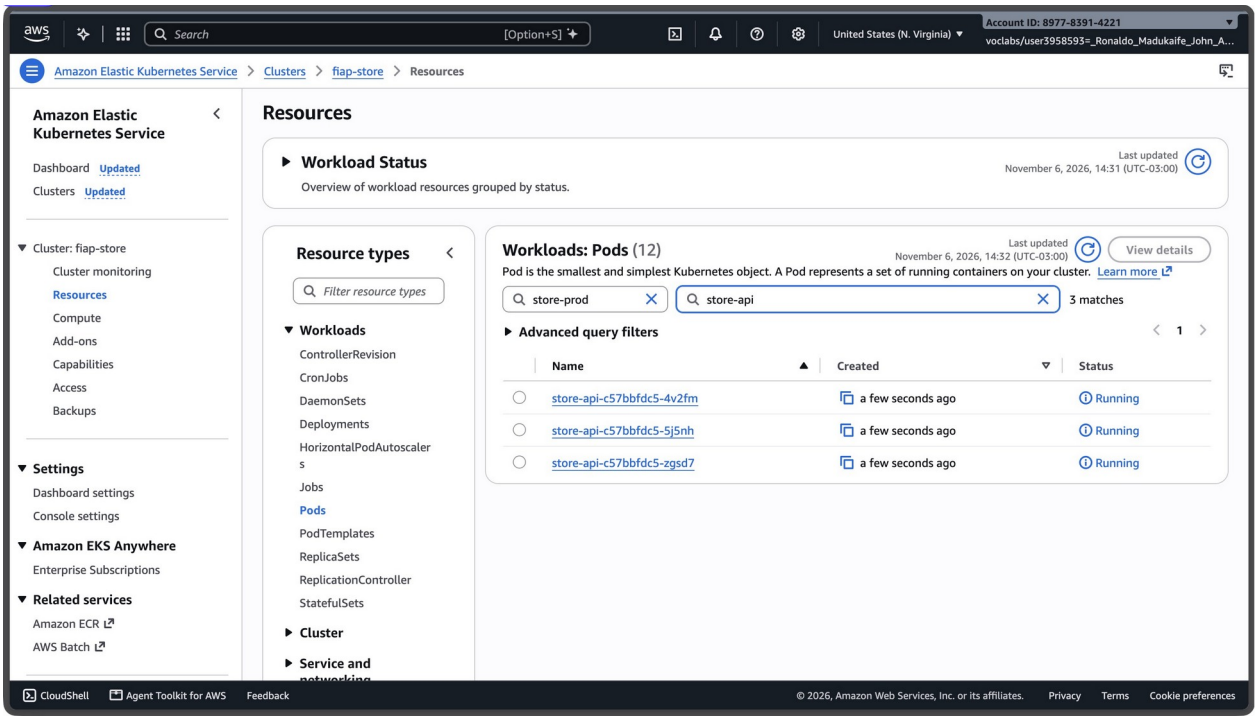


Figura 3 Três Pods da API em execução no namespace store-prod. Readiness, liveness, HPA e PDB foram verificados pelo script do professor.

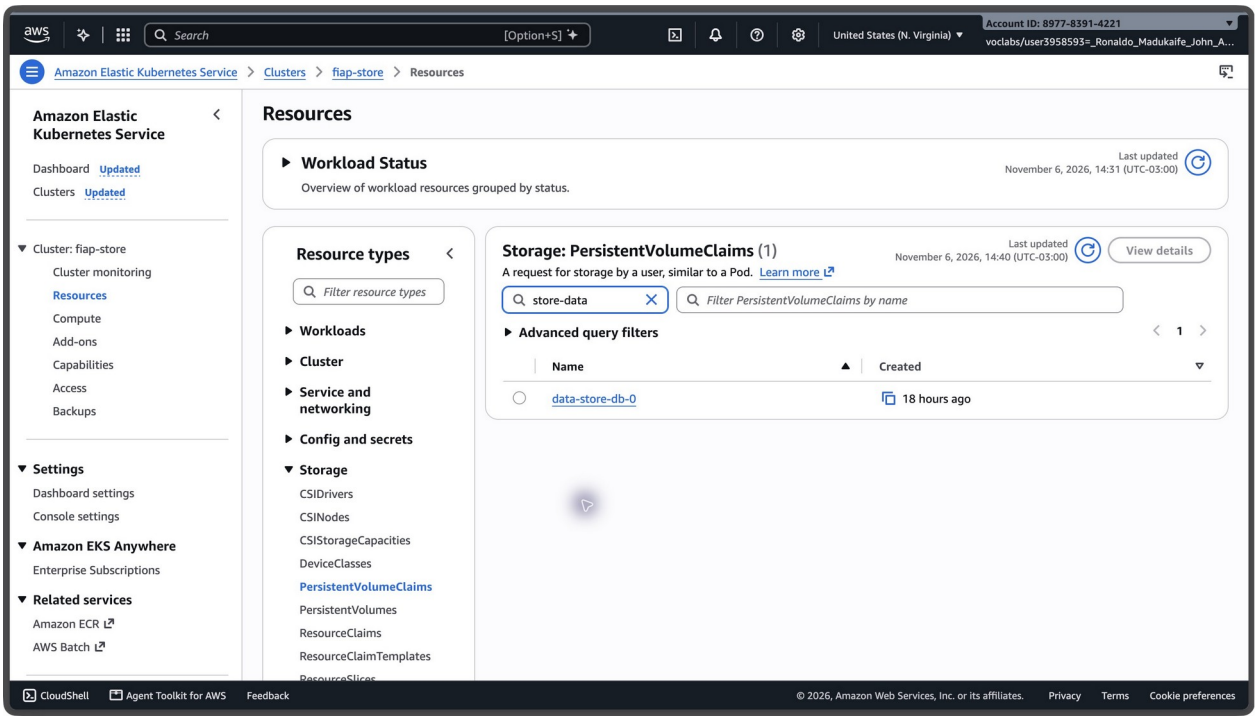


Figura 4 PVC data-store-db-0 no console. O estado Bound foi confirmado por kubectl; após recriar o Pod do banco, os mesmos 50 pedidos e o mesmo volume foram preservados.

GitOps e acesso à loja

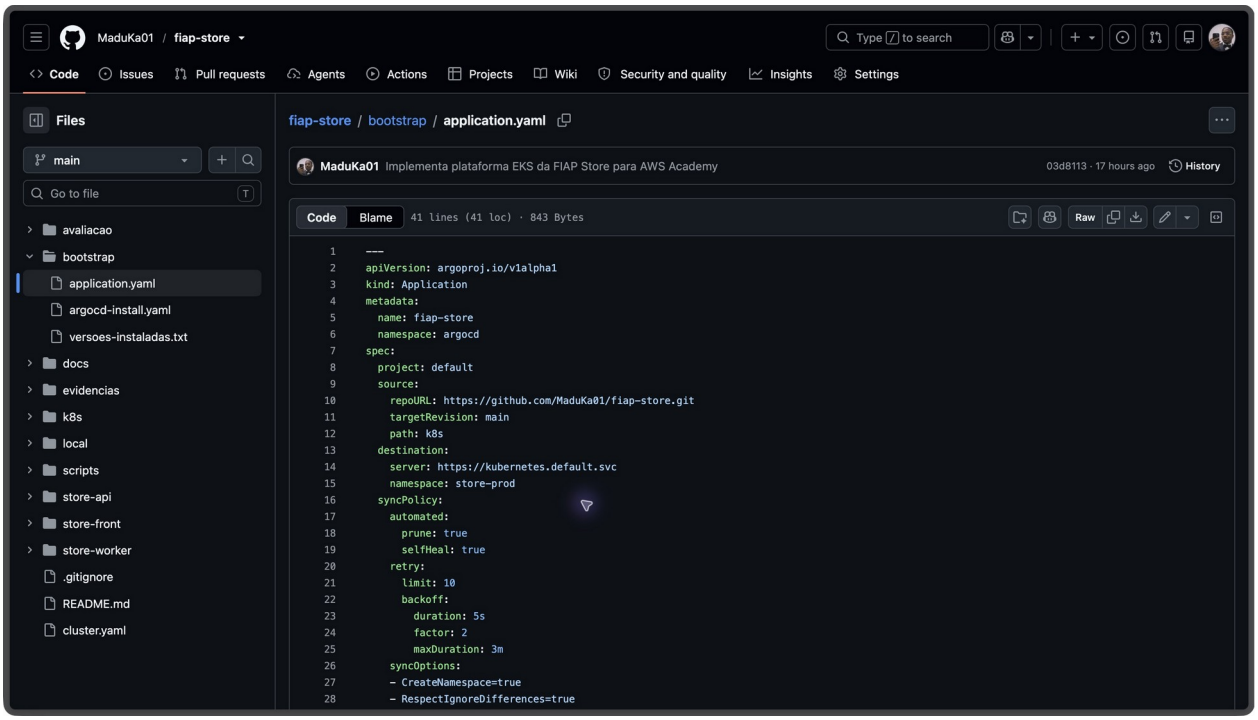


Figura 5 Manifesto da Application no GitHub com prune e selfHeal. O estado Synced/Healthy foi confirmado no cluster; o rollback e a restauração foram feitos por commits.

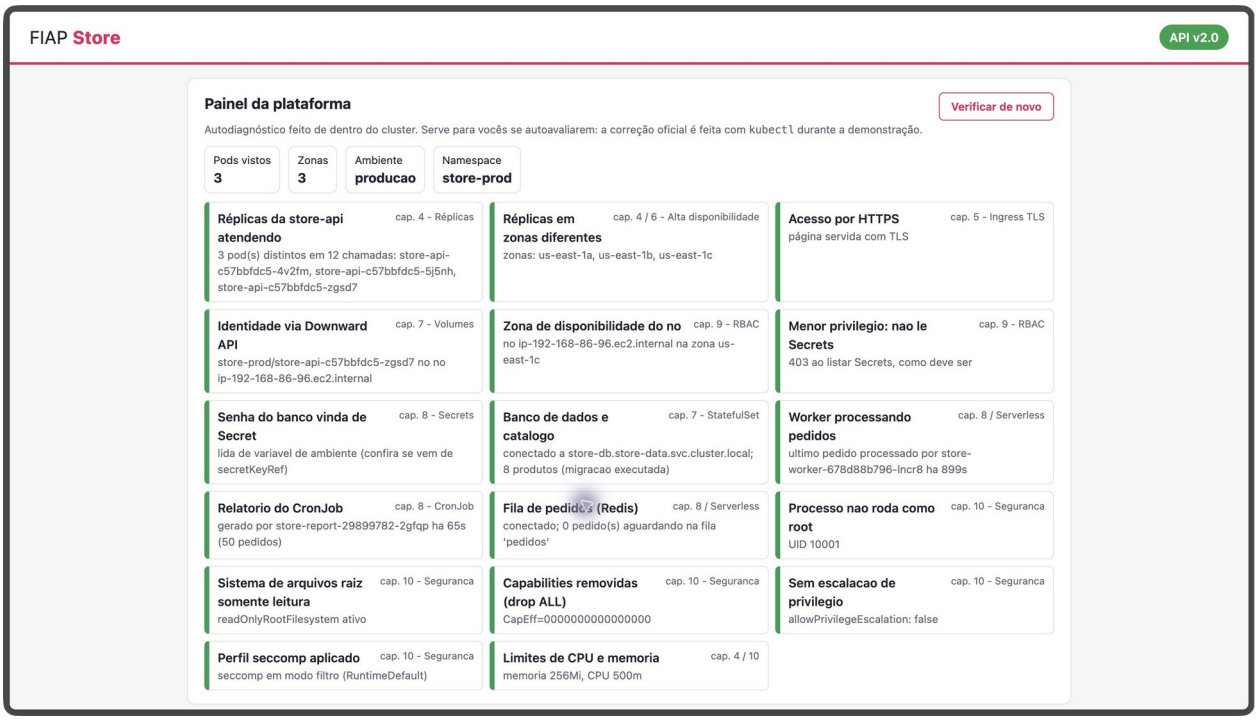


Figura 6 Loja e painel da plataforma no navegador. O ingresso público usa TLS autoassinado do laboratório, com SAN do hostname do balanceador.

Resultados do ensaio

Ensaios realizados em 06/11/2026. O processamento chegou a cinco réplicas de worker e retornou a zero. O nó drenado foi reabilitado, e a versão final da API foi restaurada para 2.0.

Desafio	Evidência observada
Self healing	Um novo Pod substituiu o removido; API voltou a responder.
Escala a zero	50 pedidos; worker fez o ciclo 0 → 5 → 0.
Relatório	Job criado do CronJob concluiu e produziu relatório.
RBAC	Pods/logs permitidos; exclusão, Secrets e exec negados.
Pod Security	Servidor recusou Pod privilegiado em dry-run por restricted.
Rede	Front sem conexão ao banco; API com conexão.
Persistência	50 pedidos e mesmo PVC/volume após recriar store-db-0.
Drenagem	Nó sem banco drenado respeitando PDBs; uncordon concluído.
Drift	Argo CD recriou o Service excluído com novo UID.
Rollback	API 1.0 e retorno a 2.0 confirmados por HTTPS após commits.

Avaliação e vulnerabilidades

Resultado automático desta sessão: 80/80. Isso não equivale à nota final de 100: README, segredos, scan e desafios ao vivo ainda são avaliados pelo professor.

Imagem	CRITICAL	HIGH
API 1.0	0	55
API 2.0	0	55
Worker 1.0	0	52
Front 1.1	0	0

Scans de 05/11/2026, Trivy 0.76.1, das mesmas imagens publicadas no ECR. O front 1.1 corrigiu quatro ocorrências CRITICAL da imagem anterior. Há achados HIGH, MEDIUM, LOW e UNKNOWN na API/worker; os JSON completos e a justificativa da correção estão versionados.

Referências da entrega

Repositório: https://github.com/MaduKa01/fiap-store
Manifestos: k8s/ | Scans: avaliacao/trivy/ | Evidências: evidencias/

Após reabrir o lab, nodeadm-config e nodeadm-run foram retomados por Systems Manager, preservando cluster e PVC. Antes de gravar, confirme sessão ativa, nós Ready e Argo CD Synced/Healthy. Criação, destruição e uso de IA constam no README.